> 🛠️ **2026.10 수정본 (v11)** — 원본: `ch05_03_03_validation(3)-ROCAUC.ipynb`
>
> - mglearn 0.2.0의 `make_blobs`가 `n_samples=(400, 50), centers=2` 조합을 못 받아서, 예전 mglearn과 **똑같은 데이터**를 만드는 함수를 노트북 안에 넣었어요.
> - 윈도우 폰트 경로 대신 공통 한글 폰트 설정(koreanize-matplotlib)을 씁니다.

평가지표 알아보기 - ROC 커브, AUC

1.1.1 이진 분류의 평가지표

1.1.2 임계값과 평가지표

1.1.3 평가지표 - ROC 커브, AUC

1.1.4 다중 분류의 평가지표

학습 내용

- 정밀도 재현율을 구하는 함수를 알아본다.
- 재현율(recall, Tprate), Fprate를 구하는 함수를 알아본다.
- 정밀도(precision), 재현율 그래프를 확인해 본다.
- Tprate, Fprate의 그래프를 확인해 본다.(ROC 커브)
- ROC커브를 구하고 AUC에 대해 이해해본다.

목차

01. 데이터 준비 및 라이브러리 임포트
02. 정밀도-재현율 곡선 확인
03. 정밀도-재현율의 커브(랜덤 포레스트 모델)
04. 평균 정밀도 확인
05. ROC 곡선과 AUC 알아보기

한글 설정

In [1]:
```
import matplotlib
from matplotlib import font_manager, rc
import platform
import warnings
warnings.filterwarnings(action='ignore')
```

In [2]:
```
# 2026.10 수정: 윈도우 폰트 경로 대신 공통 한글 폰트 설정
# 한글 폰트 설정 (코랩·윈도우·맥 공통)
%pip install -q koreanize-matplotlib setuptools
import setuptools  # 2026.10 수정: Python 3.12+엔 distutils가 없어 koreanize-ma
import koreanize_matplotlib  # 나눔고딕을 matplotlib 기본 폰트로 등록
```

```
3.11.9

Note: See 'pyenv help global' for tips on allowing multiple
      Python versions to be found at the same time.
```

01. 데이터 준비 및 라이브러리 임포트

목차로 이동하기

정밀도 재현율 곡선을 이용하여 성능을 판단해 보기

In [3]:
```
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report
```

In [4]:
```
# 2026.10 수정: 예전 mglearn의 make_blobs는 n_samples=(400, 50)처럼 클래스별 개수와
# mglearn 0.2.0부터는 sklearn의 make_blobs를 그대로 써서 이 조합이 ValueError가 납니다
# 예전과 똑같은 데이터가 나오도록 예전 mglearn 방식을 그대로 옮긴 함수입니다.
import numpy as np

def make_blobs(n_samples, centers=2, cluster_std=1.0, random_state=None,
               n_features=2, center_box=(-10.0, 10.0)):
    rng = np.random.RandomState(random_state)
    centers = rng.uniform(center_box[0], center_box[1], size=(centers, n_fea
    X, y = [], []
    for i, (n, std) in enumerate(zip(n_samples, cluster_std)):
        X.append(centers[i] + rng.normal(scale=std, size=(n, n_features)))
        y += [i] * n
    X, y = np.concatenate(X), np.array(y)
    idx = np.arange(len(y))
    rng.shuffle(idx)  # 클래스 순서를 섞기
    return X[idx], y[idx]
```

In [5]:
```
# from mglearn.datasets import make_blobs  # 2026.10 수정: 바로 위 셀의 make_blo
X, y = make_blobs(n_samples=(400, 50),
                  centers=2, cluster_std=[7.0, 2],
                  random_state=22)

print(X.shape, y.shape)
```

```
(450, 2) (450,)
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
svc = SVC(gamma=.05).fit(X_train, y_train)
```

```
tree = DecisionTreeClassifier().fit(X_train, y_train)
```

In [7]:
```
# SVC 모델
pred = svc.predict(X_test)
print(classification_report(y_test, pred))
```

```
              precision    recall  f1-score   support

           0       0.97      0.89      0.93       104
           1       0.35      0.67      0.46         9

    accuracy                           0.88       113
   macro avg       0.66      0.78      0.70       113
weighted avg       0.92      0.88      0.89       113
```

In [8]:
```
# 의사결정 트리 모델
pred = tree.predict(X_test)
print(classification_report(y_test, pred))
```

```
              precision    recall  f1-score   support

           0       0.98      0.88      0.93       104
           1       0.37      0.78      0.50         9

    accuracy                           0.88       113
   macro avg       0.67      0.83      0.71       113
weighted avg       0.93      0.88      0.90       113
```

02. 정밀도-재현율 곡선 확인

목차로 이동하기

- 새로운 모델을 만들 때, 운영을 위해 90% 재현율을 유지하는 것등의 필요조건을 지정할 때가 있다.
  - 이를 운영 포인트를 지정한다고 말하고, 운영 포인트를 고정하면 비즈니스 목표 설정이나 고객 내 다른 그룹의 성능을 보장하는데 도움이 된다.
- 이런 경우에 과제를 잘 이해하기 위해 모든 임계값을 조사하거나, 정밀도나 재현율의 모든 장단점을 살펴본다.
  - 이를 위해 정밀도-재현율 곡선을 사용

정밀도(x)와 재현율(y) - ROC 커브 확인해 보기

- precision_recall_curve() 메서드 이용

In [9]:
```
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test,
                                                       svc.decision_functior
```

In [10]:
```
# 부드러운 곡선을 위해 데이터 포인트 수를 늘립니다
X, y = make_blobs(n_samples=(4000, 500),
                  centers=2,
                  cluster_std=[7.0, 2],
                  random_state=22)

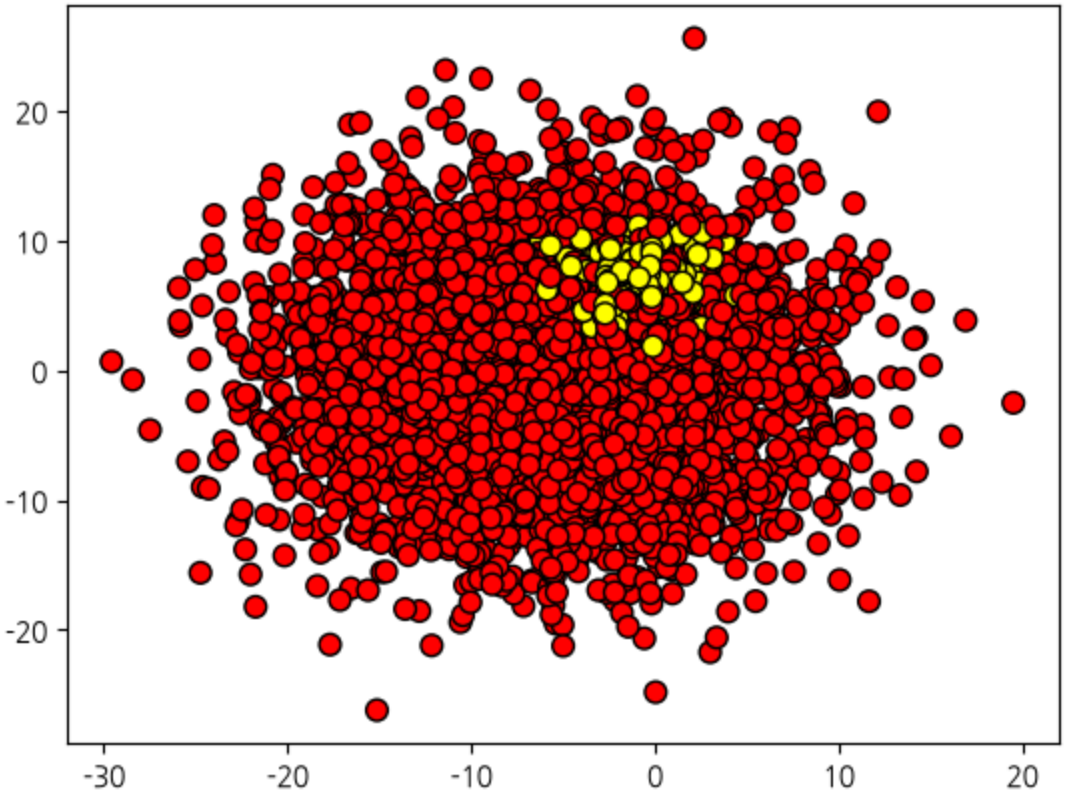
print(X.shape, y.shape)


X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=10)
```

```
(4500, 2) (4500,)
```

In [11]:
```
plt.scatter(X[:,0], X[:,1],
            c=y,
            cmap=plt.cm.autumn,  # plt.cm.Blues, RdYlGn, BrBG, Greens, RdGy,
            s=60, edgecolors='k')
```

Out[11]: <matplotlib.collections.PathCollection at 0x11681ba70>



정밀도-재현율 곡선 확인

In [12]:
```
svc = SVC(gamma=.05).fit(X_train, y_train)

pred = svc.decision_function(X_test)  #  0의 값을 기준으로 분포
print(pred[0:10])
```

```
[-1.09425577 -1.10667545 -1.10736997 -1.19140534 -1.22918652 -1.19749983
 -1.17059048 -1.24720063 -1.19551381 -1.11214569]
```

In [13]:
```
precision, recall, thresholds = precision_recall_curve(y_test, pred)

print("임계값 : ", thresholds.min(),  thresholds.max())

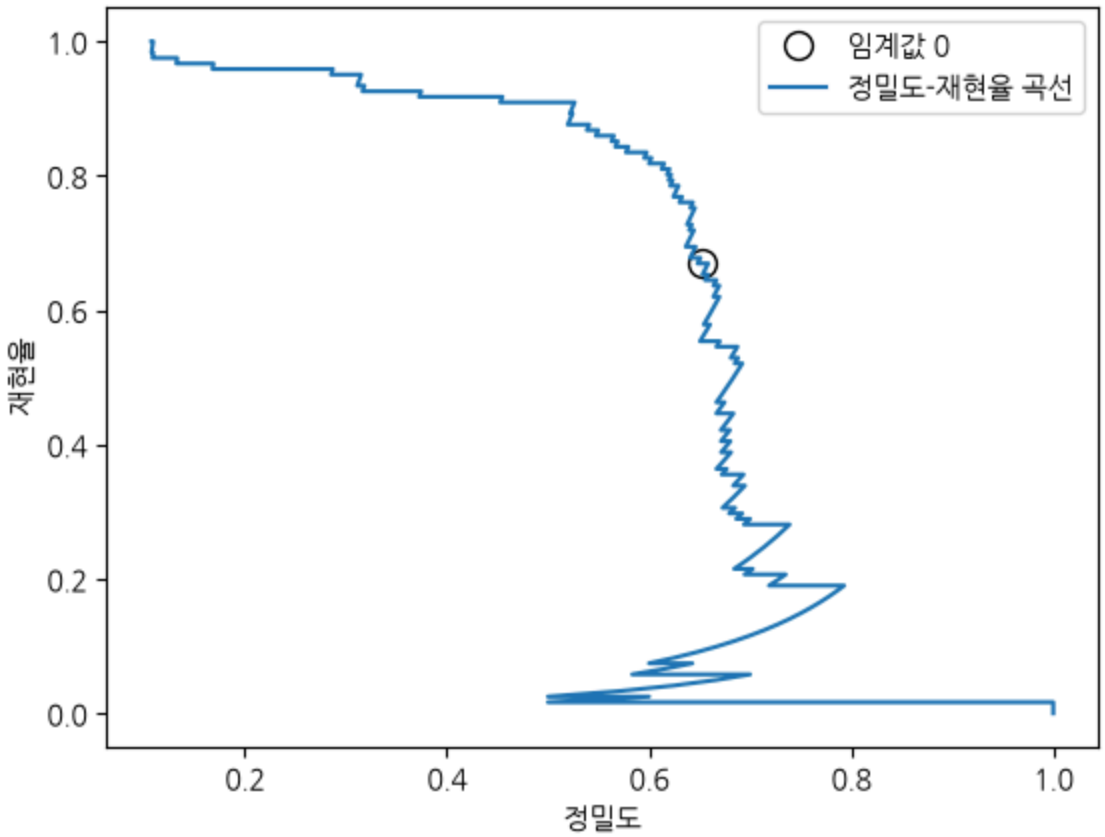
# 0에 가까운 임계값을 찾습니다
close_zero = np.argmin(np.abs(thresholds))  # thresholds의 절대값이 가장 작은 것(
print(close_zero)

plt.plot(precision[close_zero],
         recall[close_zero], 'o',
         markersize=10,
         label="임계값 0",
         fillstyle="none", c='k')

plt.plot(precision, recall, label="정밀도-재현율 곡선")
plt.xlabel("정밀도")
plt.ylabel("재현율")
plt.legend(loc="best")
```

```
임계값 :  -1.779883558959341 1.496879982456683
1001
```

Out[13]: <matplotlib.legend.Legend at 0x1168ea7e0>



정밀도

$$\text{정밀도(precision)} = \frac{\text{잘 예측(TP)}}{\text{예측을 양성으로 한 것 전체(TP+FP)}} \quad (1)$$

재현율(recall, 민감도, TPR)

- 실제 양성 데이터를 양성으로 잘 예측

$$\text{민감도(recall, 재현율)} = \frac{\text{잘 예측(TP)}}{\text{실제 값이 양성인것 전체(TP+FN)}} \quad (2)$$

가짜 양성 비율(Fprate)

- 1 - 특이도
- 실제 음성 데이터 중에 음성(0) 데이터를 잘못 예측한 비율

$$\text{가짜 양성 비율(Fprate)} = \frac{\text{틀린 예측(FP)}}{\text{실제 값이 음성인것 전체(FP + TN)}} \quad (3)$$

03. 정밀도-재현율의 커브(랜덤 포레스트 모델)

목차로 이동하기

In [14]:
```
X_train.shape  # 현재 feature 개수
```

Out[14]:
```
(3375, 2)
```

In [15]:
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=0, max_features=2
rf.fit(X_train, y_train)
pred =  rf.predict_proba(X_test)[:, 1]
pred
```

Out[15]:
```
array([0.  , 0.  , 0.  , ..., 0.  , 0.66, 0.  ], shape=(1125,))
```

SVC모델과 RandomForest모델의 비교

In [16]:
```
# SVC모델 그래프
plt.plot(precision, recall, label="svc")

plt.plot(precision[close_zero],
         recall[close_zero], 'o',
         markersize=10,
         label="svc: 임계값 0",
         fillstyle="none",
         c='k',
         mew=2)
```

```
# RandomForestClassifier는 decision_function 대신 predict_proba를 제공합니다.
precision_rf, recall_rf, thresholds_rf = precision_recall_curve(y_test, prec

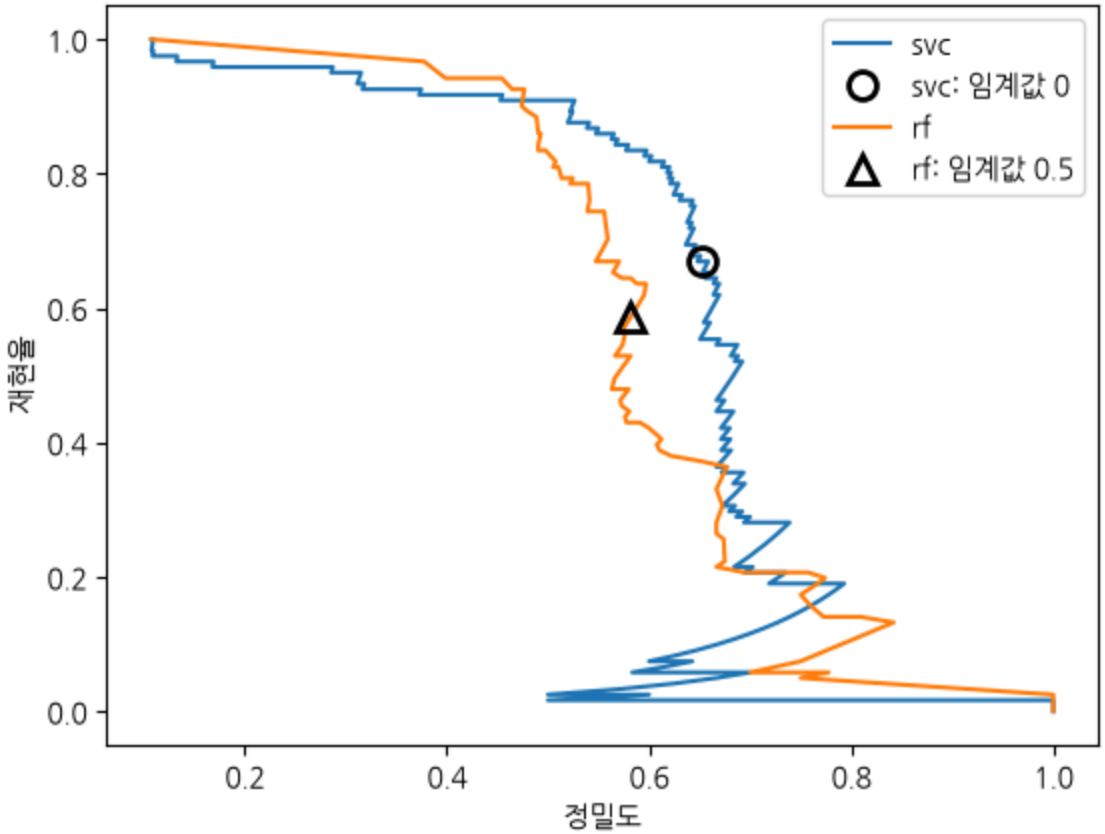
# 랜덤포레스트 그래프
plt.plot(precision_rf, recall_rf, label="rf")

close_zero_rf = np.argmin(  np.abs(thresholds_rf - 0.5)  )    # 임계값이 0.5 위
print(close_zero_rf)

plt.plot(precision_rf[close_zero_rf], recall_rf[close_zero_rf], '^', c='k',
         markersize=10, label="rf: 임계값 0.5", fillstyle="none", mew=2)
plt.xlabel("정밀도")
plt.ylabel("재현율")
plt.legend(loc="best")
```

```
46
```

Out[16]: <matplotlib.legend.Legend at 0x116903740>



- [이해하기] 정밀도 재현율 곡선이 오른쪽 위로 갈수록 더 나은 분류기를 나타낸다.
- 재현율이 매우 높거나, 정밀도가 매우 높을 때는 랜덤포레스트가 더 좋은 성능을 갖는다.
- 정밀도 0.7부분에서는 SVM이 좋음
- f1-score 점수만으로 모델의 성능을 평가한다면 이런 세세한 부분을 놓칠 수 있다.
  - f1-score는 정밀도-재현율 곡선의 **한 지점인 기본 임계값에 대한 점수**이기 때문에

(실습) 의사결정트리 모델 추가해 보기

04. 평균 정밀도 확인

목차로 이동하기

In [17]:
```
from sklearn.metrics import f1_score

rf_f1score = f1_score(y_test, rf.predict(X_test) )
svc_f1score = f1_score(y_test, svc.predict(X_test))

print("랜덤 포레스트의 f1_score: {:.3f}".format(rf_f1score))
print("svc의 f1_score: {:.3f}".format(svc_f1score))
```

```
랜덤 포레스트의 f1_score: 0.573
svc의 f1_score: 0.661
```

모델을 비교하기 위한 방법 중의 하나로 정밀도-재현율 곡선의 아랫부분 면적을 이용

- 이 면적을 평균 정밀도(average precision)이라 한다.
- 함수로 average_precision_score함수로 **평균 정밀도 계산**이 가능하다.

In [18]:
```
from sklearn.metrics import average_precision_score

## 확률 예측
rf_pro = rf.predict_proba(X_test)[:, 1]
svc_dcfun = svc.decision_function(X_test)

ap_rf = average_precision_score(y_test, rf_pro)
ap_svc = average_precision_score(y_test, svc_dcfun)

print("랜덤 포레스트의 평균 정밀도: {:.3f}".format(ap_rf))
print("svc의 평균 정밀도: {:.3f}".format(ap_svc))
```

```
랜덤 포레스트의 평균 정밀도: 0.607
svc의 평균 정밀도: 0.632
```

- 평균 정밀도는 두 모델의 성능이 비슷하다. 위의 f1_score의 결과와는 조금 다르다.

05. ROC 곡선과 AUC 알아보기

목차로 이동하기

ROC 곡선

- **ROC 곡선은 여러 임계값에서 분류기의 특성을 분석하는데 널리 사용되는 도구.**
- ROC 곡선은 분류기의 모든 임계값을 고려
- 앞의 그래프의 x는 정밀도, y가 재현율(TPR)이었다면
- ROC곡선은 x는 (False Positive rate), y를 재현율(True Positive rate)로 한것.
- 진짜 양성 비율(TPR), 거짓 양성 비율(FPR)

ROC 와 AUC

- x축은 Fprate, y축은 Tprate(재현율, 민감도)로 한다.
- FPrate는 1-특이도와 같다
- FPrate는 실제 음성인 데이터 중에 양성으로 예측하여 틀린 것의 비율

$$\text{FPRate} = \frac{\text{틀린 예측}(\text{FP})}{\text{실제 값이 음성인것 전체}(\text{FP} + \text{TN})} \tag{4}$$

In [19]:
```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, svc.decision_function(X_test))
```

임계값에 따른 각각의 Fprate, Tprate를 구하기

In [20]:
```
fpr.shape, tpr.shape, thresholds
```

Out[20]:
```
((120,),
 (120,),
 array([        inf,  1.49687998,  1.47581867,  1.4610055 ,  1.45802113,
         1.44126949,  1.39147826,  1.37380751,  1.35378001,  1.35115809,
         1.23928869,  1.22953332,  1.21983036,  1.20198557,  1.18379643,
         1.16423601,  1.03010903,  0.96014974,  0.95503679,  0.9238038 ,
         0.91992255,  0.91412964,  0.91086392,  0.91041837,  0.86674391,
         0.85989618,  0.85955523,  0.77050185,  0.76271236,  0.76034868,
         0.74948191,  0.74083693,  0.67295553,  0.6313474 ,  0.59612443,
         0.56820147,  0.5421393 ,  0.52460529,  0.51559663,  0.50719384,
         0.37819992,  0.35974527,  0.35286999,  0.35078072,  0.33633912,
         0.28457478,  0.28263032,  0.262315  ,  0.22681224,  0.22489642,
         0.20402619,  0.19841834,  0.1535657 ,  0.13174612,  0.11071806,
         0.06216232,  0.05418311,  0.05175504,  0.02982469, -0.01082579,
        -0.0441958 , -0.07263297, -0.14957616, -0.16204013, -0.22312698,
        -0.26617542, -0.28148442, -0.29640122, -0.33187486, -0.36364591,
        -0.37896216, -0.413147  , -0.41411063, -0.46562446, -0.47806116,
        -0.48767225, -0.5058179 , -0.51036328, -0.5110437 , -0.52644175,
        -0.53222803, -0.58481903, -0.58940514, -0.630053  , -0.65681141,
        -0.69947483, -0.70880328, -0.76236455, -0.76766451, -0.79133577,
        -0.80125093, -0.84469204, -0.84626488, -0.89704645, -0.89985197,
        -0.91717414, -0.93386616, -0.98339756, -0.98997147, -0.99198362,
        -0.99232122, -1.00442937, -1.00450048, -1.0275707 , -1.02764516,
        -1.05723133, -1.05811294, -1.06372408, -1.06460692, -1.09122745,
        -1.0915653 , -1.11441756, -1.11452809, -1.18688593, -1.18689626,
        -1.38761101, -1.3947979 , -1.53375669, -1.55283915, -1.77988356]))
```

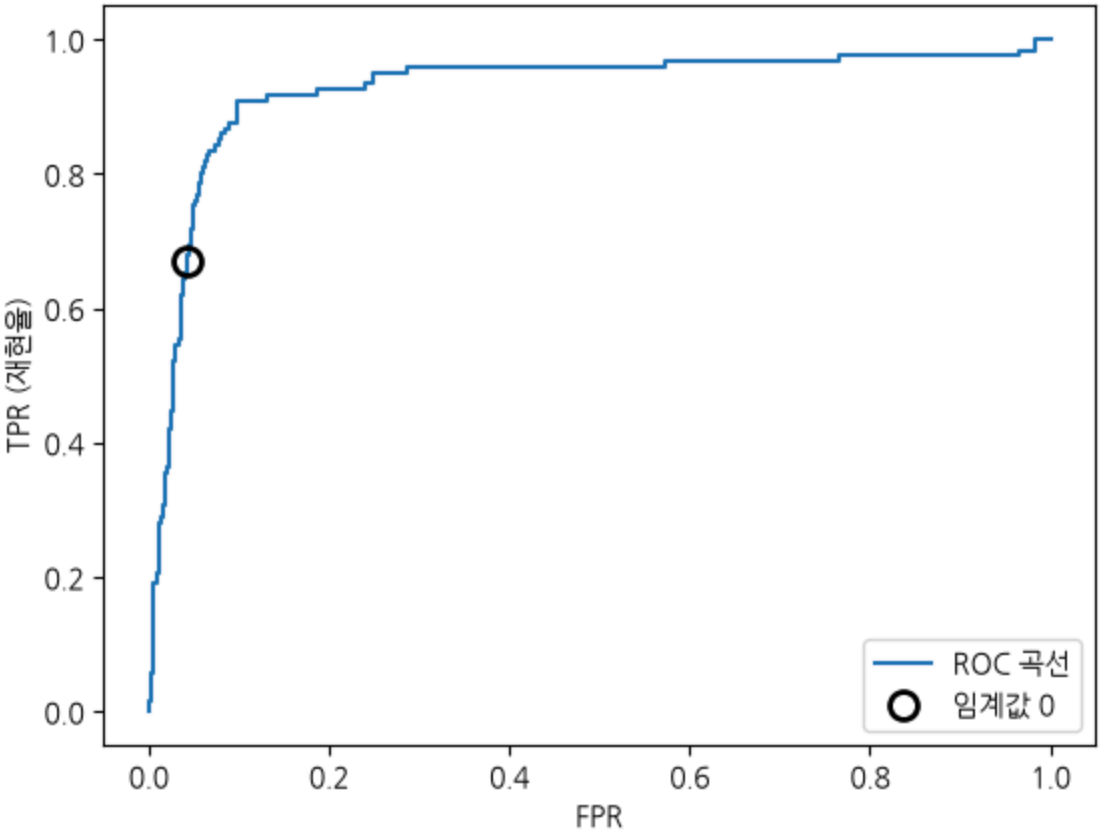
ROC곡선은 왼쪽 위쪽에 가까울 수록 성능이 좋은 것을 나타낸다.

In [21]:
```
plt.plot(fpr, tpr, label="ROC 곡선")
plt.xlabel("FPR")
plt.ylabel("TPR (재현율)")

# 임계값이 0 근처의 임계값을 찾습니다
```

```
close_zero = np.argmin(np.abs(thresholds))
plt.plot(fpr[close_zero], tpr[close_zero], 'o', markersize=10,
         label="임계값 0", fillstyle="none", c='k', mew=2)
plt.legend(loc=4)
```

Out[21]: <matplotlib.legend.Legend at 0x116e23770>



In [22]:
```
from sklearn.metrics import roc_curve
fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, rf.predict_proba(X_test)[:

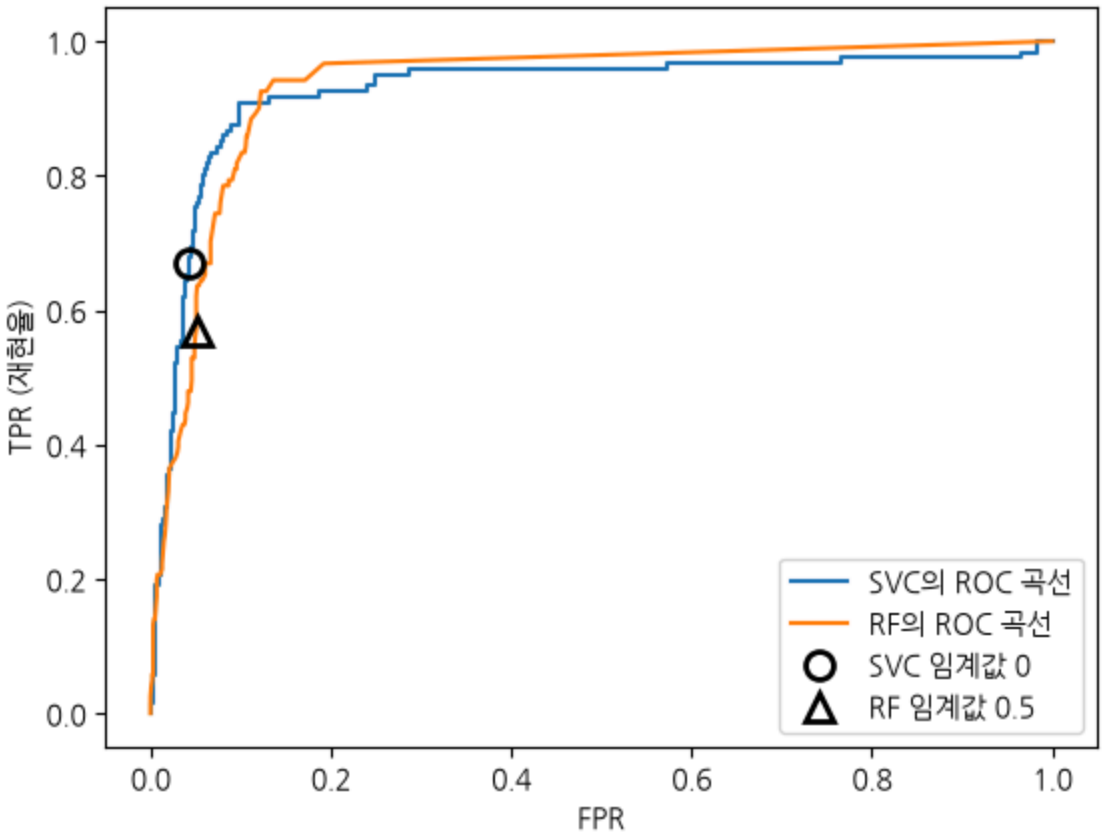
plt.plot(fpr, tpr, label="SVC의 ROC 곡선")
plt.plot(fpr_rf, tpr_rf, label="RF의 ROC 곡선")

plt.xlabel("FPR")
plt.ylabel("TPR (재현율)")
plt.plot(fpr[close_zero], tpr[close_zero], 'o', markersize=10,
         label="SVC 임계값 0", fillstyle="none", c='k', mew=2)

close_05_rf = np.argmin(np.abs(thresholds_rf - 0.5))
plt.plot(fpr_rf[close_05_rf], tpr_rf[close_05_rf],
         '^', markersize=10,
         label="RF 임계값 0.5", fillstyle="none", c='k', mew=2)

plt.legend(loc=4)
```

Out[22]: <matplotlib.legend.Legend at 0x116e80b00>



In [23]:
```
from sklearn.metrics import roc_auc_score
rf_auc = roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1])
svc_auc = roc_auc_score(y_test, svc.decision_function(X_test))

print("랜덤 포레스트의 AUC: {:.3f}".format(rf_auc))
print("SVC의 AUC: {:.3f}".format(svc_auc))
```

```
랜덤 포레스트의 AUC: 0.933
SVC의 AUC: 0.925
```

REF

- plt.cm.___ : https://chrisalbon.com/python/basics/set_the_color_of_a_matplotlib/

In [ ]:
```
```